

GridPoint

Where should the warehouse go?

32.5%

cheaper than one optimally-sited depot

₹33.0L

saved per year

37 t

CO₂ avoided per year

0.0001%

cross-validation gap

Warehouse location optimisation for Indian last-mile delivery. Decides how many warehouses to build, where each one goes, and which neighborhoods it serves — by minimising real operating cost in rupees, and showing which of its answers are proven rather than merely good.

Live application	gridpointsludge.vercel.app
Source code	github.com/sakshath7408/gridpoint
Problem statement	GRIDPOINT · Theme VECTOR · Hack-a-Matics 2026

CONTENTS

1	What it does and how to use it.....	2
2	The mathematics.....	3
3	How it compares.....	5
4	Limitations and technical summary.....	6

TEAM

Tejas	Mathematical model and Python reference implementation
Sakshath	Web application, cost model, AI component, deployment
Rajath	Brute-force verifier

Every figure in this document is produced by the committed code.

1 What it does and how to use it

An operator serves a set of neighborhoods, each with a location and a daily order volume. Three decisions follow, usually made by intuition: how many warehouses, where, and who serves whom. GridPoint answers all three by minimising cost in rupees per day.

On twelve Bengaluru zones carrying 3,880 orders a day, it replaces a single central depot at ₹27,870/day with four warehouses at ₹18,824 — 32.5% cheaper. The single depot it is compared against is itself optimally sited, so that figure is not flattered by a straw man.

In one line. Not a clustering demo with a map — a costing model that happens to be solved by a clustering algorithm, and that tells you which answers are proven.

The run

Press Optimise, or ⌘/Ctrl + Enter. Four stages, each visible as it happens:

- 1 **Place the warehouses.** Weighted k-medians finds the K points minimising order-weighted cost.
- 2 **Assign every area.** Each goes to whichever warehouse is genuinely cheapest to serve it from.
- 3 **Price the network.** Trips, fuel, driver hours and rent become one rupees-per-day figure.
- 4 **Compare and prove.** Against one depot, against k-means, against exhaustive search.

Data in

ROUTE

Sample	Seven Indian metros: Bengaluru, Delhi NCR, Mumbai, Hyderabad, Chennai, Pune, Kolkata.
Upload	Any CSV. Column names need not match ours — the AI component resolves them.
Type	Paste rows as id, lat, lon, orders. Errors reported per row as you type.

Each neighborhood needs a unique id, a valid lat/lon, and a non-negative order count. Validation failures are shown, not swallowed.

Controls

The K slider sets the number of warehouses. After a run the tool reports the cost optimum and offers a one-click jump to it — §2.4 explains why one exists.

VEHICLE	ORDERS/TRIP	FUEL L/KM	KM/H	CO ₂ G/KM
Two-wheeler	25	0.020	34	46
Three-wheeler	60	0.033	28	76
Delivery van	180	0.085	30	228
Light truck	420	0.150	26	402
Electric van	150	0.011	30	110

Four congestion profiles scale effective speed, and so driver cost per kilometre: **free flow 1.00** (night dispatch), **light 1.25** (off-peak), **peak hour 1.85** (Bengaluru 9am/6pm), **gridlock 2.60** (monsoon evening).

Fuel price, driver wage and facility rent are live inputs. Two hard constraints can be switched on — a **capacity cap** per warehouse and a **maximum service radius**. Violations are flagged on the map, not hidden.

Five demand scenarios reshape volumes so you can test whether the answer moves: today, +40% growth, suburban sprawl, urban infill, downturn.

The AI component

Real data never arrives with the column names you want. GridPoint embeds each header with Xenova/all-MiniLM-L6-v2 on WebAssembly **inside the browser** — no API key, no server, nothing leaves the machine — and matches by meaning:

INCOMING HEADER	MAPPED TO	SIMILARITY
Parcels Per Day	orders	79%
Locality Name	id	72%
Y Coordinate	lat	50%
X Coordinate	lon	39%

Those percentages are raw cosine similarity. 39% looks weak but is a correct mapping — what matters is the margin over the runner-up. If the model fails to load, the tool falls back to an alias table and says so rather than pretending it used AI.

Results out

PANEL

Headline	Percentage cheaper than one central depot, and the rupee figures behind it.
Statistics	Annual saving, CO ₂ avoided, margin over an equivalent k-means network.
Trade-off	Cost against K with the optimum starred. Click a bar to jump to that K.
Costs	Fuel, driver time, facilities, cost per order, vehicle-km, CO ₂ , robustness.
Areas	Every neighborhood, its volume, its depot, its distance.
Proof	Which claim applies at this K, and the evidence behind it.

Copy summary, copy a permalink to the exact scenario, or export the network as CSV. Before/After toggles the baseline. Light and dark themes are separately colour-validated.

2 The mathematics

Neighborhood i sits at p_i with w_i orders per day. Choose K warehouse locations — anywhere on the surface, not only at existing sites — to minimise

$$f(X) = \sum_i w_i \cdot \min_j d(p_i, x_j) + K \cdot c_{\text{facility}}$$

where d is haversine distance. This is the **weighted K-medians** problem with a facility term.

2.1 k-medians, not k-means

Delivery cost is **linear** in distance — twice as far is twice the fuel and twice the driver-hour. k-means minimises **squared** distance, which is a different objective and picks a different siting: under d^2 , one distant high-volume zone drags a warehouse toward it far harder than its real cost justifies, and everyone left behind pays.

The minimisers differ even for one facility. Squared gives the weighted centroid, in closed form. Linear gives the weighted geometric median, which has no closed form for $n \geq 3$ and must be found iteratively.

We measure the difference rather than assert it — both networks, same data, same K:

K	K-MEDIANS /DAY	K-MEANS PENALTY
2	₹23,448	+6.00%
3	₹20,521	+2.81%
4	₹18,824	+2.07% optimum
5	₹19,125	+1.22%

The wrong objective costs 1.2–6.0% of total network cost, permanently, for no benefit — up to ₹5.5 lakh a year on this dataset, thrown away by reaching for the more familiar algorithm. Measured on the delivery budget alone, which is the part siting can actually move, the gap is 3.6–8.1%.

2.2 The cost model is free

For neighborhood i served from warehouse w :

$$\begin{aligned}
 \text{roadKm}_i &= d(p_i, w) \times \rho & \rho &= 1.32 \text{ detour factor} \\
 \text{trips}_i &= \lceil w_i / Q \rceil & Q &= \text{vehicle capacity} \\
 \text{vehicleKm}_i &= 2 \times \text{roadKm}_i \times \text{trips}_i & & \text{round trips} \\
 r &= \phi \cdot P_{\text{fuel}} + u + W/v_{\text{eff}} & & \text{₹ per vehicle-km} \\
 \text{cost}_i &= d(p_i, w) \times [2 \cdot \text{trips}_i \cdot \rho \cdot r]
 \end{aligned}$$

The bracketed term is **constant in w** . So minimising rupees is still exactly weighted k-medians — the whole vehicle, fuel, traffic and wage model collapses into the weights. Five fleets and four congestion profiles cost the optimiser nothing.

Stated openly: we price the trunk leg plus facilities. Distribution inside a neighborhood is excluded because it is near-invariant to depot position and cannot move the optimum.

2.3 What is proven, and what is not

K = 1 — proven globally optimal. Each $\|x - p_i\|$ is a norm, hence convex; a non-negative weighted sum of convex functions is convex; so any local minimum is the global one. Weiszfeld's iteration

$$x^{(m+1)} = \sum_i (w_i p_i / \|x^{(m)} - p_i\|) / \sum_i (w_i / \|x^{(m)} - p_i\|)$$

is a descent method on that objective, so its fixed point is the global minimum. GridPoint is not reporting the best location it found — it is reporting the best that exists.

Caveat. The iteration is undefined if an iterate lands exactly on a demand point. We floor the denominator away from zero; a production version should use the Vardi-Zhang modification.

K > 1 — NP-hard, so we certify instead. Two things are proven rather than one thing overclaimed:

- **Never worse than exhaustive search.** All $C(n,K)$ ways of siting K depots on the demand points are enumerated exactly — 495 subsets at $n=12$, $K=4$ — and warm-start the solver. Lloyd's decreases monotonically, so the result cannot be worse. That is a guarantee.
- **Certified locally optimal.** Each warehouse is perturbed across a 21×21 lattice within 2 km — 1,764 probes at $K=4$ — and none improves the objective. Certified to ~100 m, finer than a lease.

2.4 How many warehouses

Delivery cost falls with K ; rent rises linearly. The sum has a genuine interior minimum.

K	DELIVERY	FACILITIES	TOTAL/DAY	VS ONE DEPOT
1	₹25,370	₹2,500	₹27,870	—
2	₹18,448	₹5,000	₹23,448	15.9%
3	₹13,021	₹7,500	₹20,521	26.4%
4	₹8,824	₹10,000	₹18,824	32.5%
5	₹6,625	₹12,500	₹19,125	31.4%
6	₹5,174	₹15,000	₹20,174	27.6%
8	₹2,704	₹20,000	₹22,704	18.5%

A true interior minimum — $K=5$ is ₹301/day worse, not better. The curve turns. At $K=4$: ₹33.0 lakh saved a year, 37 t CO_2 avoided.

2.5 Robustness and cross-validation

Demand resampled 16 times at $\pm 30\%$, comparing keeping today's siting against re-optimising with perfect hindsight. Mean regret 0.64%, worst case 2.77%. Since relocating a warehouse costs far more than 0.64% of daily operating cost, the correct decision is to build and not re-site.

The model was implemented twice independently — Python on a tangent plane, TypeScript on the sphere, different seeding, different arithmetic:

K	PYTHON (ORDER-KM)	TYPESCRIPT	GAP	WORST SITE
1	55,490.66	55,490.74	+0.000%	0.000 km
3	30,378.14	30,378.17	+0.000%	0.002 km
5	17,904.77	17,904.80	+0.000%	0.000 km

Agreement to 0.0001% on cost and ~2 m on position. Reproducible with `npm run crossval`. Two independent implementations converging is the strongest evidence that the result is a property of the mathematics, not of one codebase.

3 How it compares

Not against enterprise network-design suites, which do far more — against what a team would realistically reach for.

APPROACH	WHAT IT GETS WRONG	GRIDPOINT
Intuition / existing sites	Anchored on property already leased.	Searches the continuous plane.
k-means clustering	Squared distance — 1.2–6.0% dearer at equal K.	Minimises the actual linear cost.
Centre-of-gravity sheet	Weighted centroid — k-means renamed.	Cost model, constraints, optimum in K.
Generic solver	A number, with no account of confidence.	Proven at K=1, certified above.
Enterprise network design	Powerful, but days to model a question.	An answer in under a second.

Why the mathematics holds up

- **The objective matches the cost.** Linear because fuel and driver-hours are linear — and we quantify what choosing otherwise costs.
- **Proven where provable, certified where not.** K=1 by convexity; K>1 is NP-hard, so we certify — and say which applies in the interface.
- **An exhaustive benchmark.** All C(n,K) discrete sitings enumerated as a floor the answer cannot fall below.
- **K is derived, not assumed.** The infrastructure–delivery trade-off produces a genuine interior minimum.
- **Robustness measured, not claimed.** 0.64% mean regret under $\pm 30\%$ demand movement.
- **Independently cross-validated.** Two languages, two geometries, agreeing to 0.0001%.

4 Limitations and technical summary

Stated because a model whose limits are not stated should not be trusted.

- **Great-circle $\times 1.32$** stands in for road distance — a routing API would add a live dependency we refused.
- **Independent radial trips.** Multi-drop routing would lower absolute cost but affects all sitings alike, so the argmin largely holds.
- **Uniform facility cost.** Real rents vary by location. This is the most valuable extension.
- **K > 1 is certified, not proven.** Repeated because it is the claim most likely to be over-read.
- **Static demand within a run.** Scenarios model shifts between runs, not intra-day variation.
- **Sample volumes are illustrative.** Coordinates are real, validated locality centroids; order counts are invented and labelled so.

ITEM	DETAIL
Algorithm	Weighted k-medians; Lloyd's with a geometric-median (Weiszfeld) update, k-means++ seeding, warm-started from the exact discrete p-median
Distance	Haversine $\times 1.32$ road detour factor
Cost model	Fuel + upkeep + driver time on the trunk leg, plus flat facility cost per day
AI component	Xenova/all-MiniLM-L6-v2 via Transformers.js on WebAssembly, in-browser
Stack	Next.js 15, React 19, TypeScript, MapLibre GL JS; Python 3 reference
Architecture	Entirely client-side — no backend, no database, no API keys
Tests	70 passing across geometry, cost model and all eight bonus features
Cross-validation	Python vs TypeScript to 0.0001% on cost, ~ 2 m on position